Disclaimer: These slides can include material from different sources. I'll be happy to explicitly acknowledge a source if required. Contact me for requests.

Machine Learning in a Nutshell

15-288 Spring '26

Lecture 19:
Logistic Regression Classifier

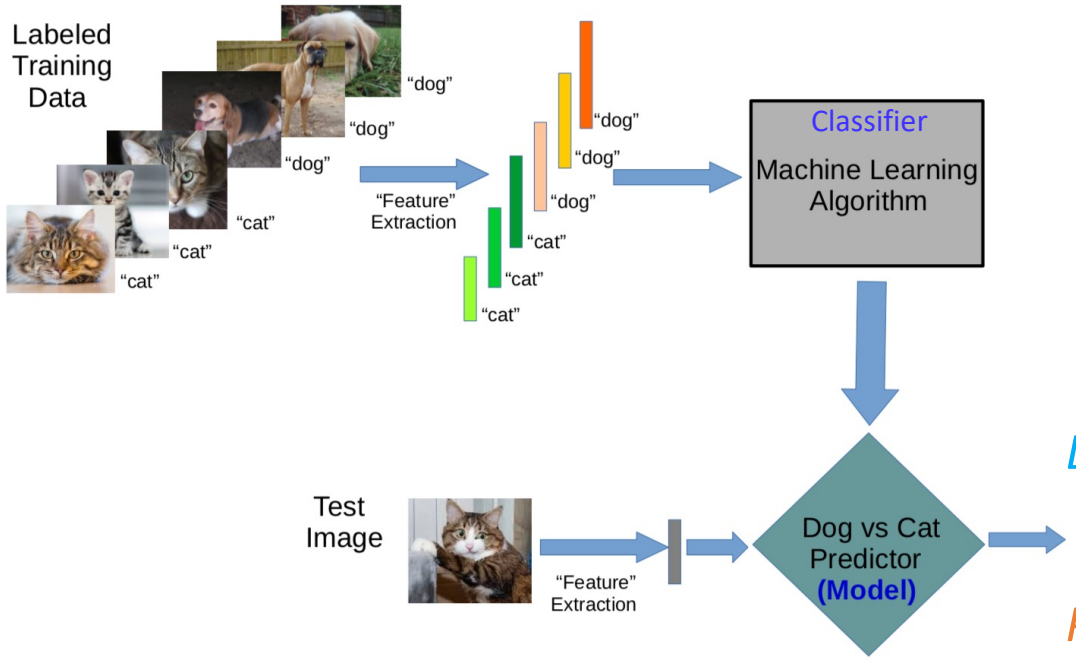
Teacher:
Gianni A. Di Caro

جامعة كارنيجي ميلون في قطر
Carnegie Mellon University Qatar

Probabilistic view: ML as Probability Density Estimation

- **Probabilistic formulation of learning:**
  - Don't want (just) to learn a function approximator of $f: X \rightarrow Y$ to **predict** $y \in Y$ given $\boldsymbol{x} \in X$
  - Want to learn the **probability** p that the label is $y \in Y$ given that the input is $x \in X$:

conditional probability density: $p(y \mid \boldsymbol{x})$



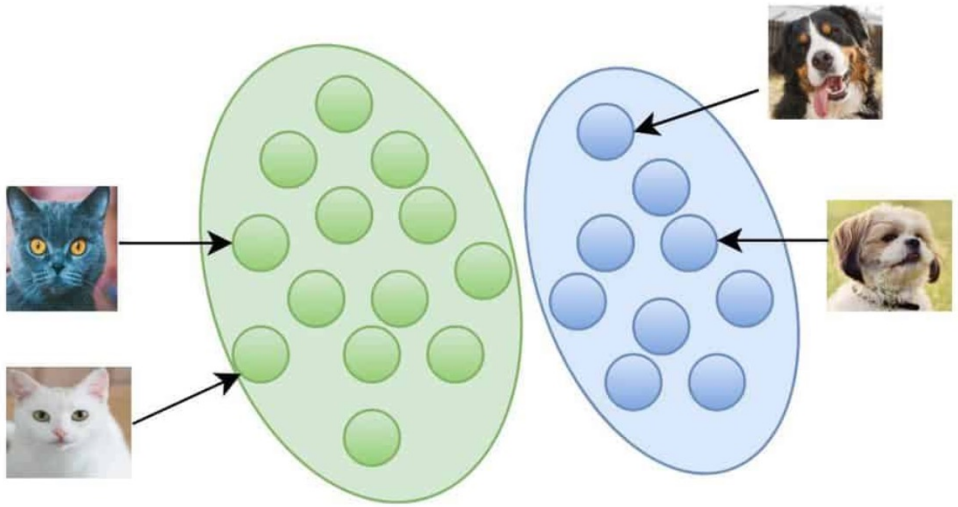
Deterministic model: Output **Label** → either Cat or Dog

Probabilistic model: Output **Label** probability

(e.g., 0.82 to be a cat, 0.18 to be a dog)

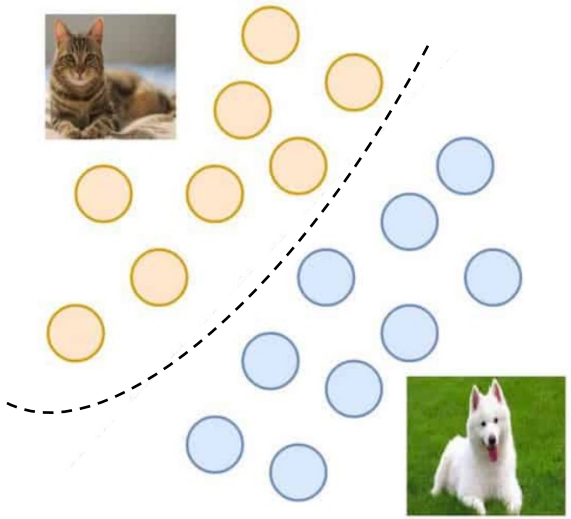
Probabilistic view: Generative vs. Discriminative modeling

- The conditional probability density $p(y \mid \boldsymbol{x})$ can be **estimated**:
  - ✓ In two steps, passing by <u>first</u> estimating the full joint distribution (**generative model**)
  - ✓ Or, directly, in one-step (**discriminative model**)



Generative

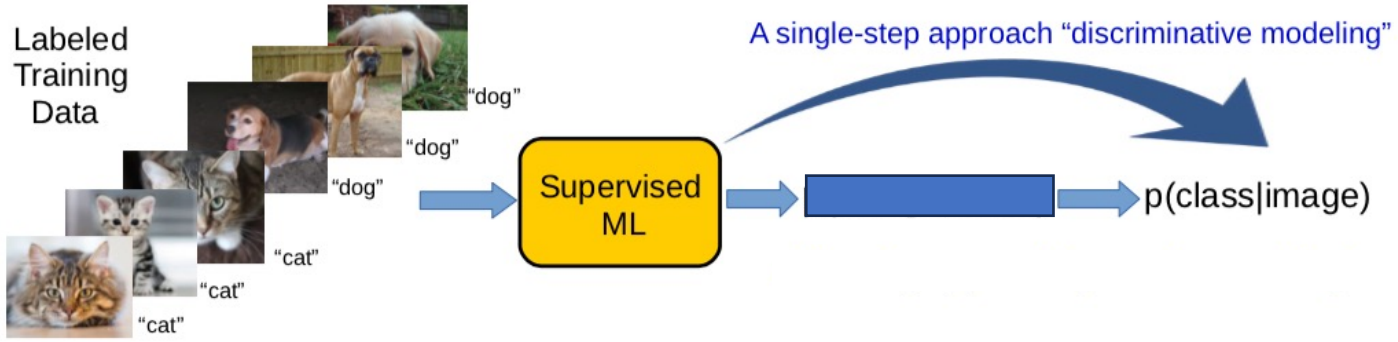
Learn the <u>concept</u> of a cat and of a dog
→ Can *generate* (new) cats and dogs



Discriminative

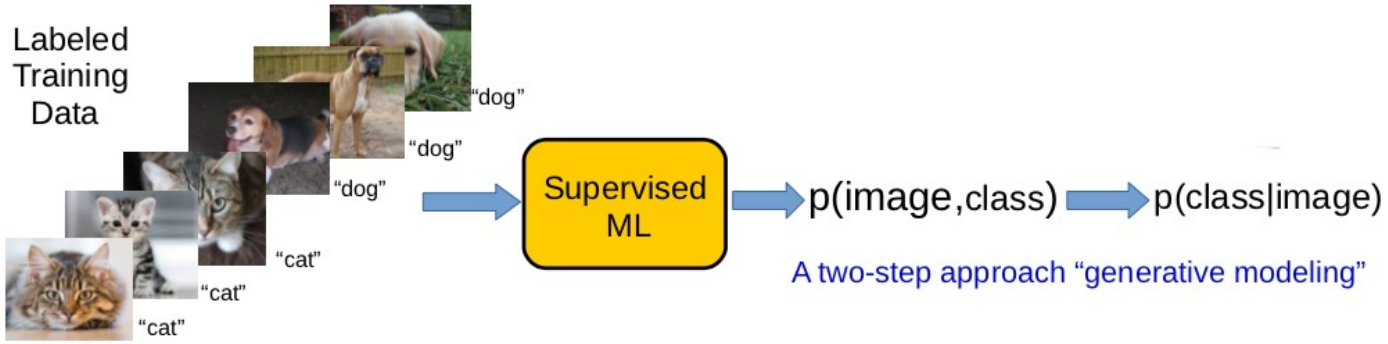
Learn to *discriminate* between cats and dogs

Probabilistic view: Generative vs. Discriminative modeling



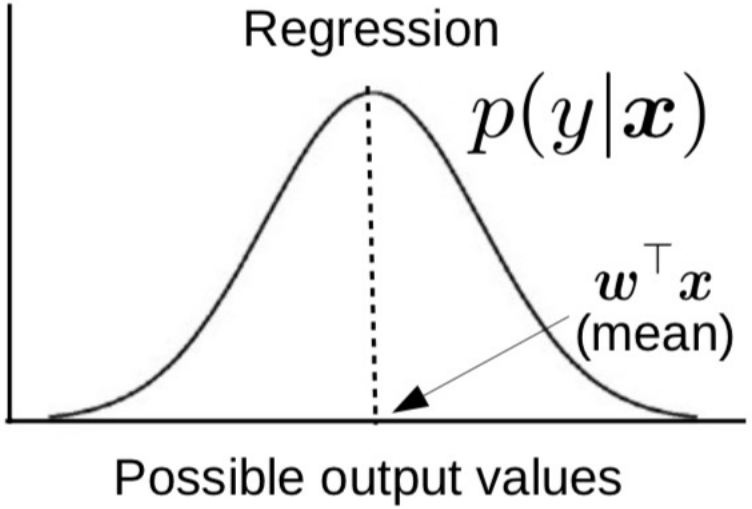
Bayes rule!

$$P(y \mid \boldsymbol{x}) = \frac{p(\boldsymbol{x} \mid y)P(y)}{p(\boldsymbol{x})} = \frac{p(\boldsymbol{x}, y)}{p(\boldsymbol{x})}$$

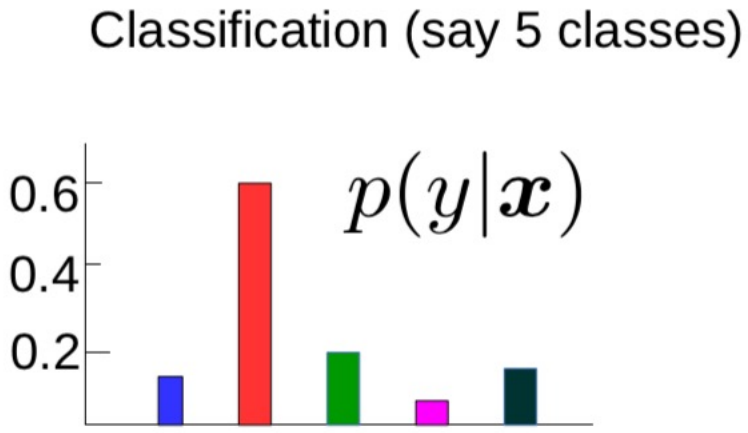


Probabilistic view → Learn Conditional Probability Distributions

- **Function approximation view**: learn the mapping $f: X \to Y$ that predicts y given an input $\boldsymbol{x}$
- **Probabilistic view**: Learn the conditional probability distribution $p(y \mid \boldsymbol{x})$, that gives the probability of output y given an input $\boldsymbol{x}$, or learn the joint distribution $p(y, \boldsymbol{x})$ and use it to compute $p(y \mid \boldsymbol{x})$



(conditional probability *density* function, **PDF**)



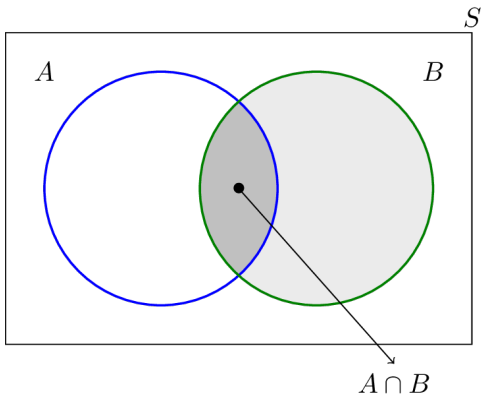
(conditional probability *mass* function, **PMF**)

Conditional (probability) distributions

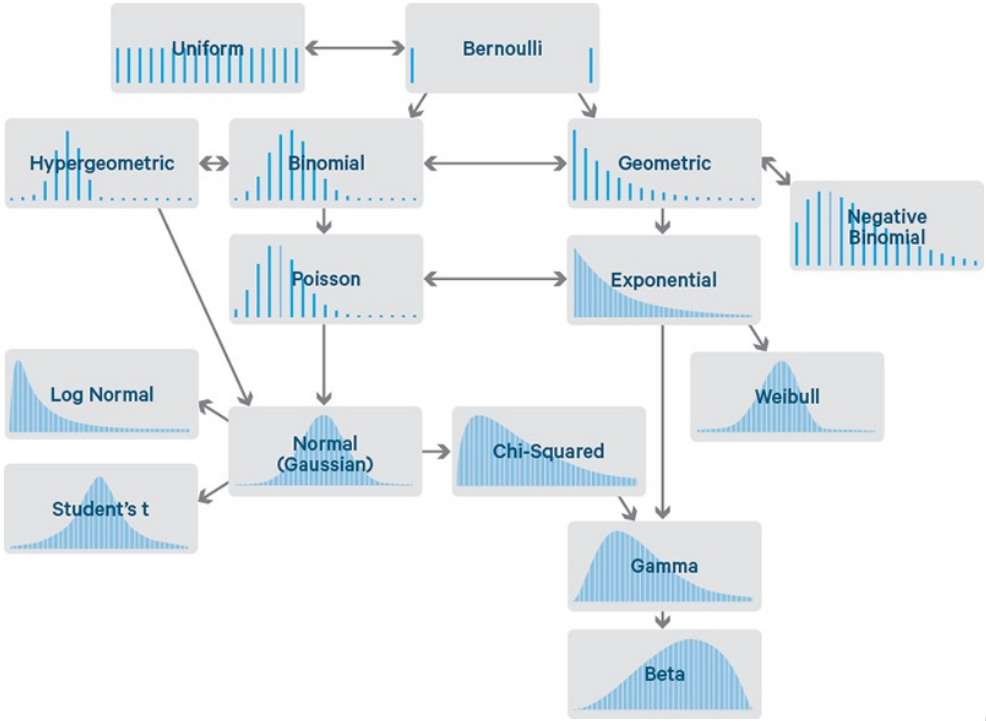
Probabilistic view: pros and cons

- The distribution $p(y \mid \boldsymbol{x})$ of possible outputs is more informative compared to mapping $f: X \rightarrow Y$
  - ✓ Gives the *expected* or most likely value of the predicted output y
  - ✓ Can be used to quantify the *uncertainty* in the predicted output y
  - • Gives *soft predictions* (e.g., rather than yes/no prediction, it gives probability of answer)

- 🥶 **We have to deal with probabilities, conditional probabilities, Bayes rule, probability distributions …**



$$P(A|B) = \frac{P(A \cap B)}{P(B)}$$



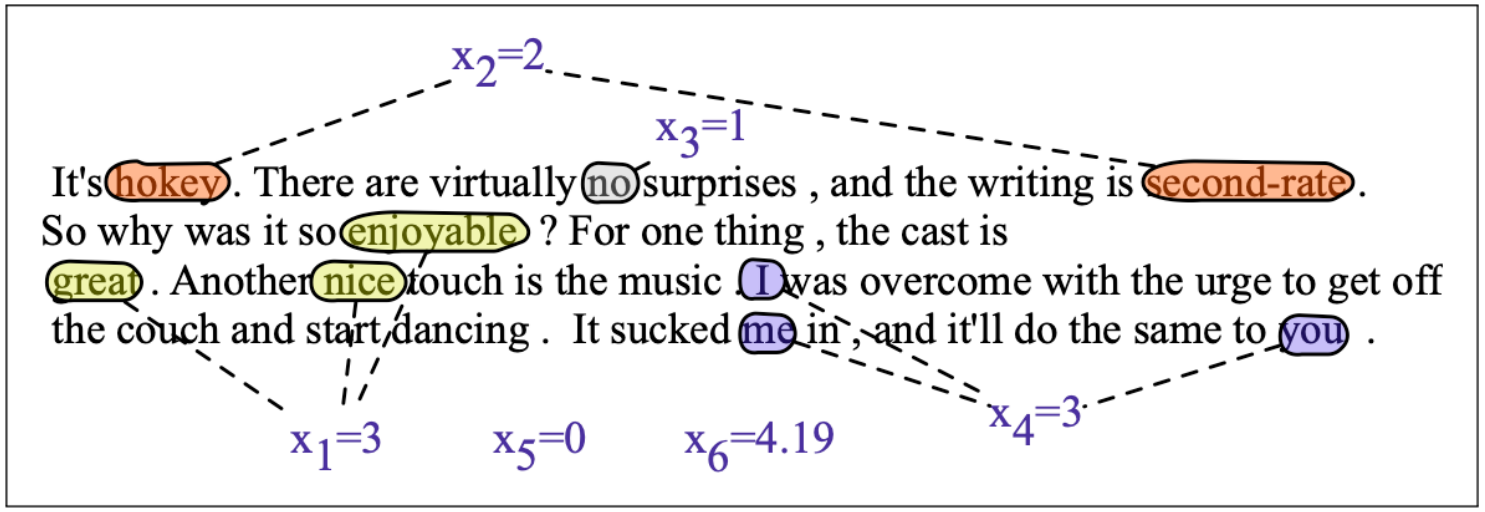
Discriminative Linear classifier, $p(y|\boldsymbol{x})$

Goal: A probabilistic linear classifier that learns a discriminative model, i.e., $p(y|\boldsymbol{x})$

- E.g., **binary** sentiment classification on movie review text, and we would like to know whether to assign the sentiment class + or − to a review document *doc*.
  - Since sentiment is not necessarily something that can be measured always with precisions from a text, we prefer to have a probabilistic classifier, that assigns a probability to output labels
  - We have 6 features, and we have a way to label a word as positive or negative

➢ What is the probability of $y = +\ sentiment$ for the given doc, i.e., given $\boldsymbol{x} = (x_1, x_2, x_3, x_4, x_5, x_6)$?

Var	Definition	Value
x_1	count(positive lexicon words $\in$ doc)	3
x_2	count(negative lexicon words $\in$ doc)	2
x_3	$\begin{cases} 1 & \text{if "no"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	1
x_4	count(1st and 2nd pronouns $\in$ doc)	3
x_5	$\begin{cases} 1 & \text{if "!"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	0
x_6	ln(word count of doc)	$\ln(66) = 4.19$

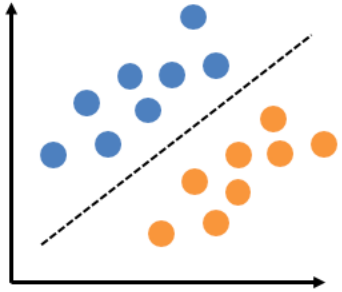


This and other examples are from: *Speech and Language Processing*, D. Jurafski, and J.H. Martin, 2025

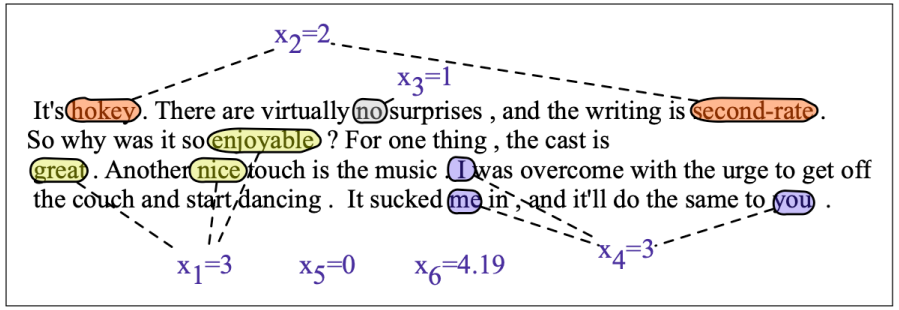
Discriminative Linear classifier, $p(y|\boldsymbol{x})$

Goal: A probabilistic linear classifier that learns a discriminative model, i.e., $p(y|\boldsymbol{x})$

- (Binary) Linear classifier, $f(\boldsymbol{x})$:
  - ✓ Is defined as a **linear function of the features**: $f(\boldsymbol{x}) = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d$
  - ✓ Defines a **linear decision boundary** between the classes (in the raw feature space, with no kernelization/transformations)



- (Binary) Probabilistic classifier, $f(\boldsymbol{x})$:
  - ✓ $f(\boldsymbol{x}): X \rightarrow [0,1]$
  - E.g., $f(\boldsymbol{x}) = 0.7$ means that the predicted label is class + with a probability of 0.75



$p(+|\,(x_1, x_2, x_3, x_4, x_5, x_6) = 0.7$

Logistic regression as discriminative Linear classifier

- (Binary) Linear classifier, $f(\boldsymbol{x})$:
  - ✓ Is defined as a **linear function of the features**: $f(\boldsymbol{x}) = w_0 + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d$

$$z = b + w_1 x_1 + w_2 x_2 + \cdots + w_d x_d$$
$$= \sum_{i=1}^{d} w_i x^i + b = \boldsymbol{w} \cdot \boldsymbol{x} + b$$

z is the **score** of the classifier

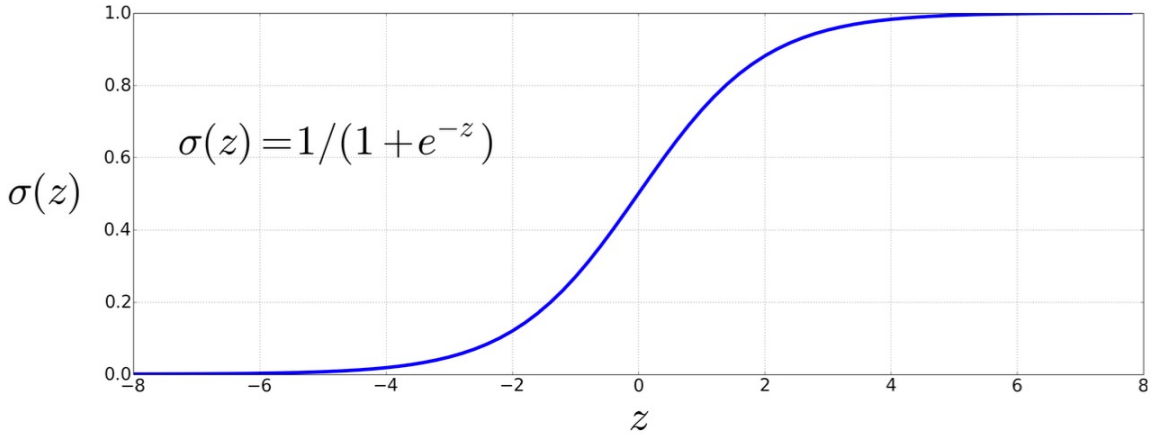
$b \equiv w_0$

- (Binary) Probabilistic classifier, $f(\boldsymbol{x})$:
  - ✓ $f(\boldsymbol{x}): X \to [0,1]$

How can we transform the score $z \in [-\infty, +\infty]$ to be a probability?

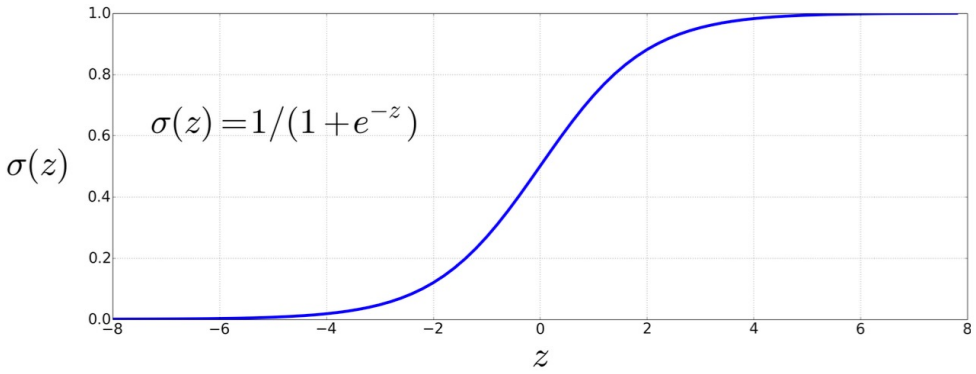
→ Transform it using the **sigmoid** function! (also, **Logistic function**)

$$\boxed{\sigma(z) = \frac{1}{1 + e^{-z}}}$$



Logistic regression: Nice properties

$$\sigma(z) = \frac{1}{1 + e^{-z}}$$



Nice properties:

- It takes a real-valued number z and maps it into the range $(0,1)$, which is just what we want in practice for a **probability**.
- it is nearly linear around 0, guaranteeing a proportional mapping between inputs and probabilities
- Because it flattens toward the ends, it tends to squash outlier values toward 0 or 1.
- It's differentiable, which useful for learning (with gradient methods)

Logistic regression for computing $p(y|\boldsymbol{x})$

$$z = \boldsymbol{w}^T\boldsymbol{x} + b \qquad \boxed{\sigma(z) = \frac{1}{1+e^{-z}}}$$

- If we apply the sigmoid to the sum z of the weighted features, we get a number between 0 and 1. To make it a conditional probability, we just need to make sure that the two cases, $p(y=1)$ and $p(y=0)$, sum to 1 → Normalize!

When the score is positive → probability of class 1 (+)
When the score is negative→ probability of class 0 (-)

$$P(Y = y \mid X = \boldsymbol{x}, \boldsymbol{w}, b) \sim$$

$$P(y = 1 \mid \boldsymbol{x}, \boldsymbol{w}, b) = \sigma(\boldsymbol{w}^T\boldsymbol{x} + b) = \frac{e^{\boldsymbol{w}^T\boldsymbol{x}+b}}{1+e^{\boldsymbol{w}^T\boldsymbol{x}+b}}$$

$$P(y = 0 \mid \boldsymbol{x}, \boldsymbol{w}, b) = 1 - \sigma(\boldsymbol{w}^T\boldsymbol{x} + b) = \frac{1}{1+e^{\boldsymbol{w}^T\boldsymbol{x}+b}}$$

Logistic regression for computing $p(y|\boldsymbol{x})$

$$\boxed{\sigma(z) = \frac{1}{1+e^{-z}}}$$

The sigmoid has the property:

$$1 - \sigma(z) = \sigma(-z)$$

$$\rightarrow P(y = 0 \;\; \boldsymbol{x}, \boldsymbol{w}, b) = \sigma(-z) = \frac{e^{-(\boldsymbol{w}^T \boldsymbol{x} + b)}}{1 + e^{-(\boldsymbol{w}^T \boldsymbol{x} + b)}}$$

$$\begin{aligned} P(y=1) &= \boldsymbol{\sigma}(\mathbf{w} \cdot \mathbf{x} + b) \\ &= \frac{1}{1+\exp(-(\mathbf{w} \cdot \mathbf{x} + b))} \end{aligned}$$

Other way to express the probabilities:

$$\begin{aligned} P(y=0) &= 1 - \boldsymbol{\sigma}(\mathbf{w} \cdot \mathbf{x} + b) \\ &= 1 - \frac{1}{1+\exp(-(\mathbf{w} \cdot \mathbf{x} + b))} \\ &= \frac{\exp(-(\mathbf{w} \cdot \mathbf{x} + b))}{1+\exp(-(\mathbf{w} \cdot \mathbf{x} + b))} \end{aligned}$$

Terminology: a few facts

- Misnomer, logistic *regression* is a model for classification!
  - The name comes from the **logistic function** $\sigma(z)$, which was extensively used in **regression models** for estimating population growths in ecosystems.

- The input to the sigmoid function, the score z, is often called the **logit**. This is because the **logit function** is the inverse of the sigmoid.
  - The logit function is the log of the odds ratio, $\frac{p}{1-p}$:

$$\text{logit}(p) = \sigma^{-1}(p) = \ln \frac{p}{1-p}$$

  - Using the term logit for z is a way of reminding us that by using the sigmoid to turn z (which ranges in$[-\infty, +\infty]$) into a probability, we are implicitly interpreting z as not just any real-valued number, but as specifically a *log* odds

Classification with logistic regression

- How do we **make a decision** about which class to apply to a test instance $\boldsymbol{x}$?
- ✓ For a given $\boldsymbol{x}$, we say Class 1 (+) if the probability $P(y = 1|\boldsymbol{x})$ is more than .5, and no otherwise.

$$decision(\boldsymbol{x}) = \begin{cases} 1 & \text{if } P(y = 1|\boldsymbol{x}) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

0.5 determines the **decision boundary**

Decision boundaries for Logistic Regression

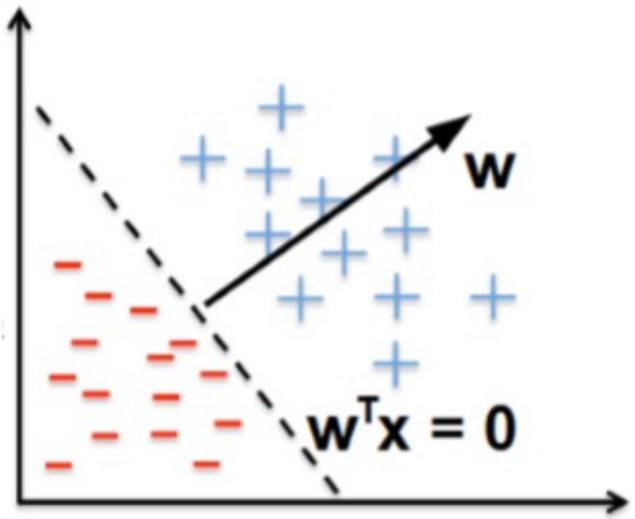
- Decision boundaries for Logistic Regression?
  - At the decision boundary, labels $1/0$ are equiprobable

$$P(y=1 \mid \boldsymbol{x}, \boldsymbol{w}, b) = \frac{1}{1+e^{-(\boldsymbol{w}^T\boldsymbol{x}+b)}} \qquad P(y=0 \mid \boldsymbol{x}, \boldsymbol{w}) = \frac{1}{1+e^{\boldsymbol{w}^T\boldsymbol{x}+b}}$$

to be equal: $e^{-\boldsymbol{w}^T\boldsymbol{x}} = e^{\boldsymbol{w}^T\boldsymbol{x}}$ whose only solution is $\boldsymbol{w}^T\boldsymbol{x} = 0$

✓ $\Longrightarrow$ Decision boundary is linear

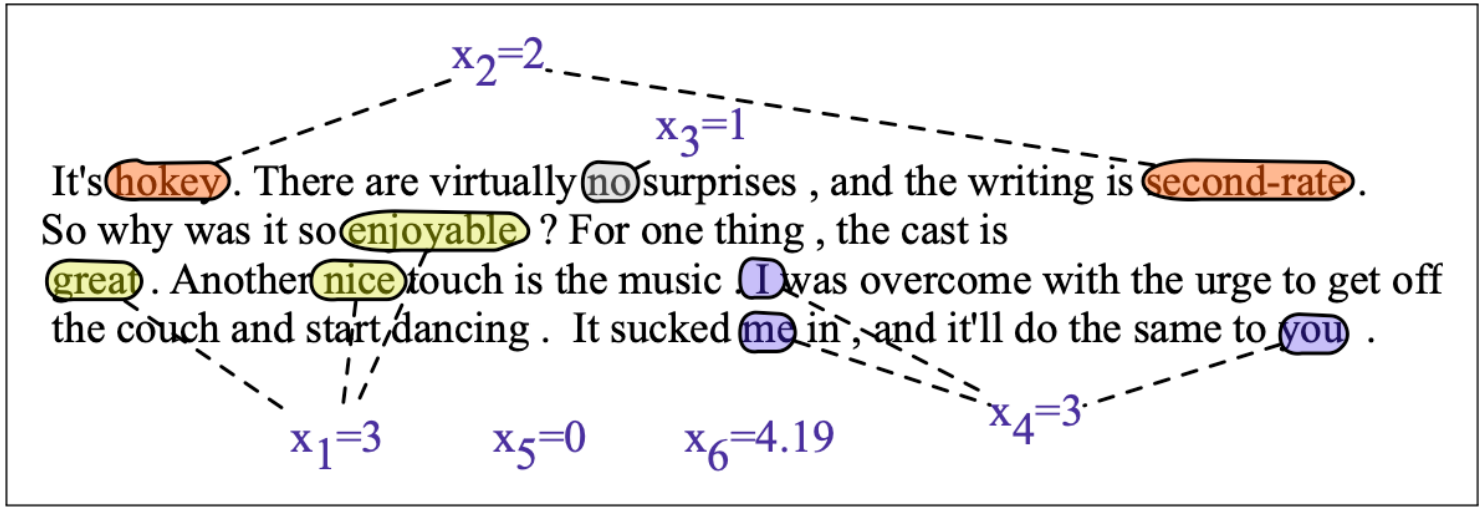
✓ $\Longrightarrow$ Logistic regression is a <u>probabilistic linear classifier</u>



Classification with logistic regression

- Let's see an example, using the sentiment classification task considered before

Var	Definition	Value
x_1	count(positive lexicon words $\in$ doc)	3
x_2	count(negative lexicon words $\in$ doc)	2
x_3	$\begin{cases} 1 & \text{if "no"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	1
x_4	count(1st and 2nd pronouns $\in$ doc)	3
x_5	$\begin{cases} 1 & \text{if "!"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	0
x_6	ln(word count of doc)	$\ln(66) = 4.19$



Assume that the weight vector has been learned: $\boldsymbol{w} = [.5, -5.0, -1.2, 0.5, 2.0, 0.7], b = 0.1$

$$\begin{aligned} p(+|x) = P(y=1|x) &= \sigma(\mathbf{w}\cdot\mathbf{x}+b) \\ &= \sigma([2.5,-5.0,-1.2,0.5,2.0,0.7]\cdot[3,2,1,3,0,4.19]+0.1) \\ &= \sigma(.833) \\ &= 0.70 \end{aligned}$$

$$\begin{aligned} p(-|x) = P(y=0|x) &= 1-\sigma(\mathbf{w}\cdot\mathbf{x}+b) \\ &= 0.30 \end{aligned}$$

→ Predicted sentiment is positive!

How do we find the best weights of the classifier?

✓ We know how to **apply** LR based on the linear input z, the sigmoid function, the decision threshold:

$$z = \boldsymbol{w}^T \boldsymbol{x} + b \qquad \boxed{\sigma(z) = \frac{1}{1 + e^{-z}}} \qquad decision(\boldsymbol{x}) = \begin{cases} 1 & \text{if } P(y = 1|\boldsymbol{x}) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

➢ How do we find the *best* values for the weights and the bias?

1. We still need to define a loss function, $\ell \rightarrow$ *What* we want to optimize
2. Then, we can effectively apply gradient methods to the linear classifier (if the loss function is *nice*)

How do we find the best weights of the classifier?

We still need to define a loss function, $\ell \rightarrow$ *What* we want to optimize

In general, we need a loss function that expresses, for an observation x, how *close* the classifier output $\hat{y} = \sigma(\boldsymbol{w}^T \boldsymbol{x} + b)$ is to the true label $y \in \{0,1\}$:

$$\ell(\hat{y}, y)$$

Since the classifier is probabilistic, we must rely on **probabilistic notions** to quantify the difference/error $\ell(\hat{y}, y)$

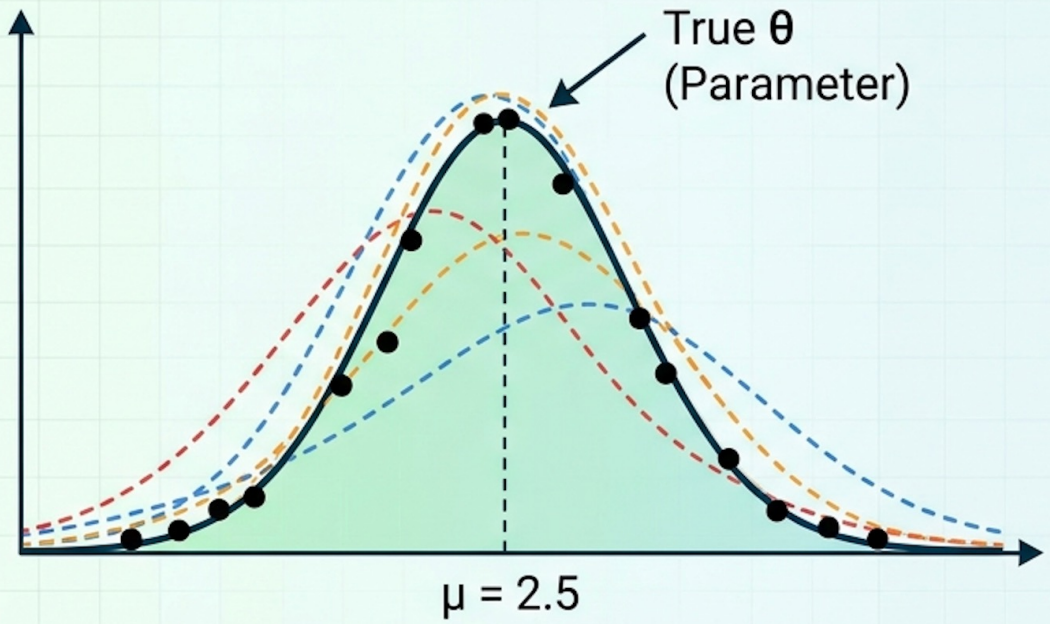
We *shall* make a little detour and consider the more general problem of **Maximum Likelihood Estimation** of the parameters of a distribution …

Note: The next Two slides shows a very high-level summary of the general problem of **Maximum Likelihood Estimation (MLE)** of the parameters of a distribution.

Slides 22 to 28 present the problem with mathematical details. These are included for completeness but are not really required. You are anyway invited to go through them to grasp a better understanding of the concepts.

MLE in a Nutshell

Maximum Likelihood Estimation (MLE): The Core Concept



Find the parameter (θ) that makes the observed data *most probable*.

Likelihood Function
$L(\theta|\text{Data}) = P(\text{Data}|\theta)$

Maximize this function

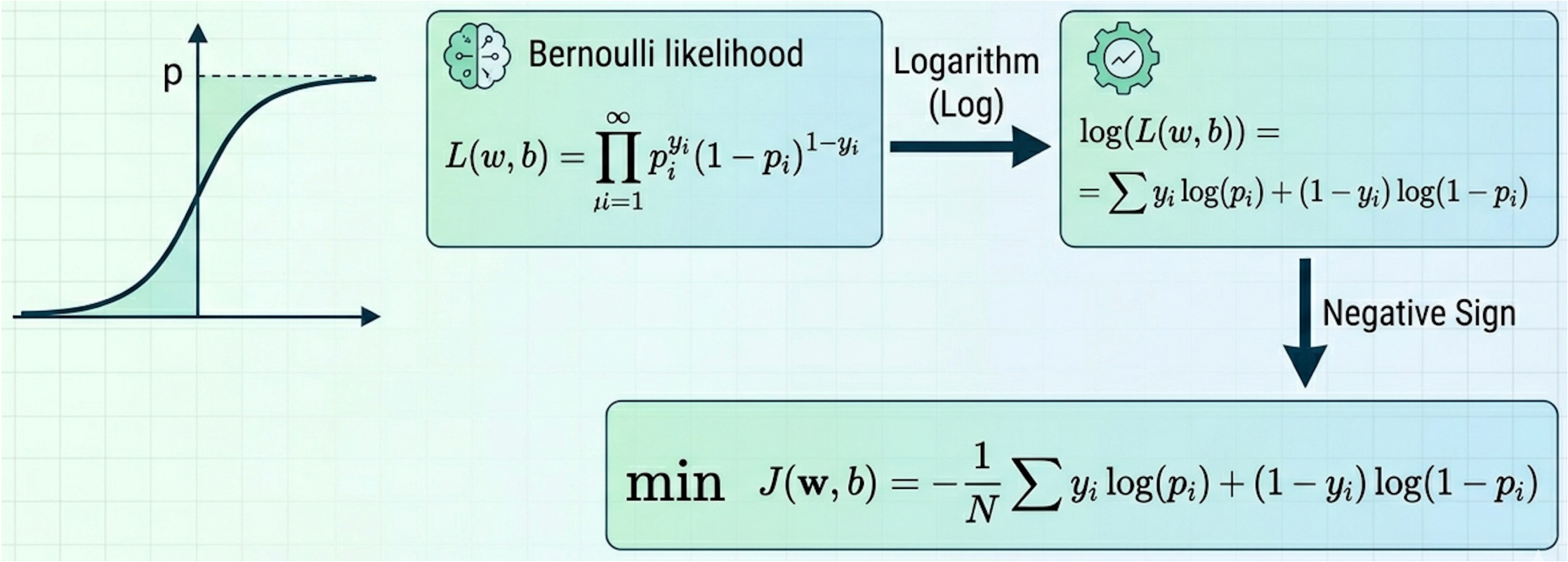
Log-Likelihood
$L^*(\theta) = \log(L(\theta))$

Typically maximized for computational ease

MLE for LR in a Nutshell

E.g., labels in $\mathcal{D}$ are {1, 1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0}

❖ **Bernoulli distribution**: $p(y_n \mid \theta) = \theta^{y_n}(1-\theta)^{1-y_n}$



Probabilistic view of a labeled dataset: Bernoulli trials

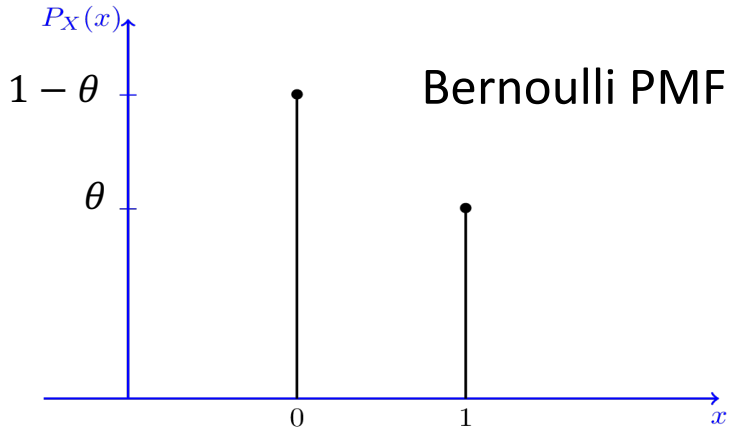
- Let's consider a **binary classification task**, where $y \in \{0, 1\}$

E.g., labels in $\mathcal{D}$ are {1, 1, 0, 0, 1, 1, 0, 1, 0, 1, 0, 0, 0, 1, 0, 1, 1, 1, 0, 1, 1, 1, 1, 0}

❖ **Parametric assumption**: Let $\mathcal{D}$ model data from repeated **Bernoulli trials** (e.g., coin tosses): observations from random experiments whose outcome is either 0 or 1

$\mathcal{D}$ = {H, H, T, T, H, H, T, H, T, H, T, T, T, H, T, H, H, H, T, H, H, H, H, T}

- In general, the coin could be *biased* (unfair) such that not necessarily the probability θ of getting H (1) vs. T (0) is exactly 0.5 → It needs to be **estimated from the data**: $\hat{\theta}$?



Bernoulli PMF

❖ **Bernoulli distribution**: $p(y_n \mid \theta) = \theta^{y_n}(1-\theta)^{1-y_n}$

$$p(1 \mid \theta) = \theta^1(1-\theta)^0 = \theta$$

$$p(0 \mid \theta) = \theta^0(1-\theta)^1 = 1-\theta$$

Coin toss example: MLE estimation of Bernoulli distribution

$\mathcal{D}$ = {H, H, T, T, H, H, T, H, T, H, T, T, T, H, T, H, H, H, T, H, H, H, H, T}

- What would be your choice for $\hat{\theta}$?

- Solution (frequentist answer): use *frequencies of occurrences* to do the estimate :

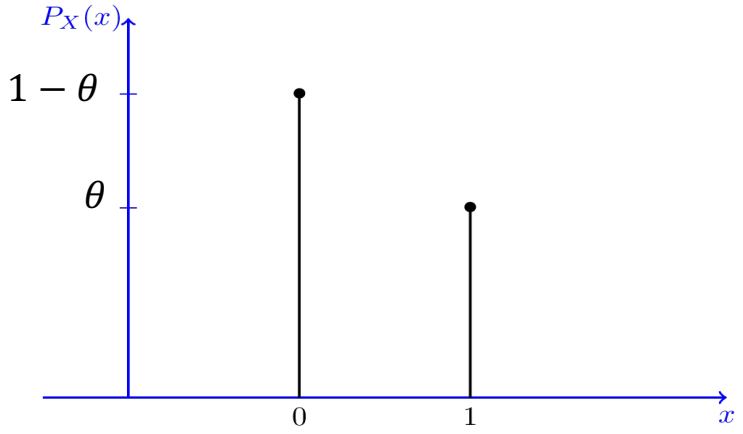
$$\hat{\theta} = \frac{\#H}{\#H + \#T} = \frac{15}{25} = 0.6$$

- This is indeed the Maximum Likelihood Estimate (MLE) answer! → Let's get it formally

Coin toss example: MLE estimation of Bernoulli distribution

- Each outcome is a random variable y_i that can take one of two values, $\{0, 1\}$
  - Each y_i is a **binary random variable**

➢ The PMF model that we can adopt is the Bernoulli distribution



❖ **Bernoulli distribution**: $p(y_n \mid \theta) = \theta^{y_n}(1-\theta)^{1-y_n}$

○ **Parametric distribution**, where the (unknown) parameter θ corresponds to the probability of the random variable being 1:

$$p(y_n = 1 \mid \theta) = \theta^1(1-\theta)^{1-1} = \theta$$

○ <u>Goal</u>: learn θ using the MLE criterion (with Log-likelihood)

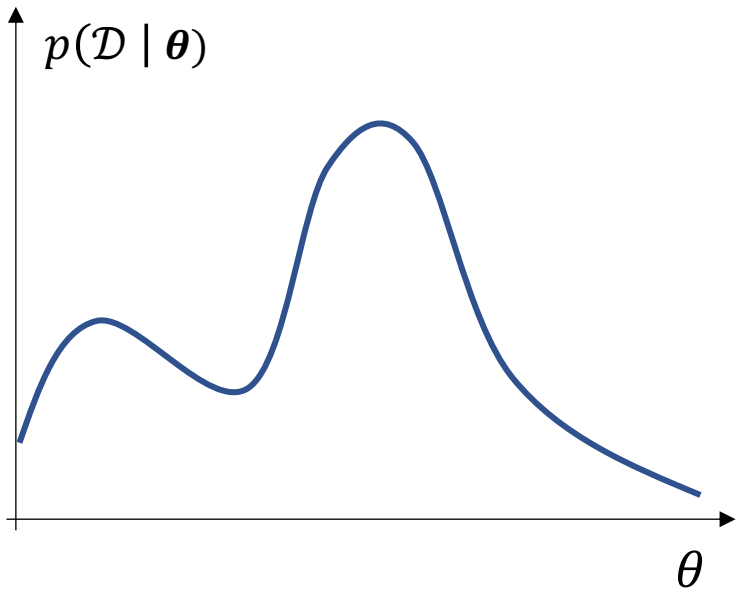
Likelihood function

- What is the probability of the observed data **given** a parameter vector $\boldsymbol{\theta}$?
  - i.e., IF the coin is biased according to the probability parameter $\boldsymbol{\theta}$ (e.g., $\boldsymbol{\theta}$=0.8), what is the probability of observing the coin labels present in $\mathcal{D}$?

$$P(\mathcal{D} \mid \boldsymbol{\theta}) = p(y_1, y_2, \cdots, y_N \mid \boldsymbol{\theta})$$

✓ Data is assumed i.i.d., therefore, the probability of *jointly* observing the samples in $\mathcal{D}$ is the **product** of the probability of observing each sample individually:

$$P(\mathcal{D} \mid \boldsymbol{\theta}) = \prod_{i=1}^{N} p(y_i \mid \boldsymbol{\theta})$$



❖ $P(\mathcal{D} \mid \boldsymbol{\theta})$ is the **likelihood of the model**: how likely is to observe the data $\mathcal{D}$ if the data were generated by the probabilistic model described by the assumed density function p and the parameter vector $\boldsymbol{\theta}$

- The likelihood is a function of $\boldsymbol{\theta}$
- $p(y_i \mid \boldsymbol{\theta})$ is the likelihood of a single data sample

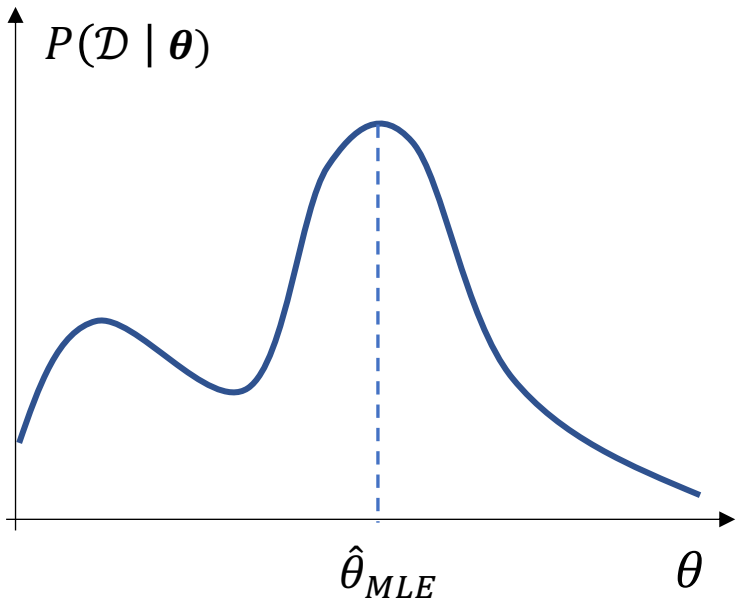
Maximum Likelihood Estimation (MLE)

- <u>Goal:</u> find the *best* estimate $\hat{\boldsymbol{\theta}}$ of the true $\boldsymbol{\theta}$ from the data $\mathcal{D}$

- ✓ **<u>MLE criterion</u>**: find the value of $\boldsymbol{\theta}$ that maximizes the probability of the observed data in $\mathcal{D}$
  (the data in $\mathcal{D}$ are most likely to be observed for that (best) value of $\boldsymbol{\theta}$)

  - Maximize the *Likelihood* $P(\mathcal{D} \mid \boldsymbol{\theta})$ of $\mathcal{D}$ with respect to $\boldsymbol{\theta}$ → **Maximum Likelihood Estimate (MLE)**

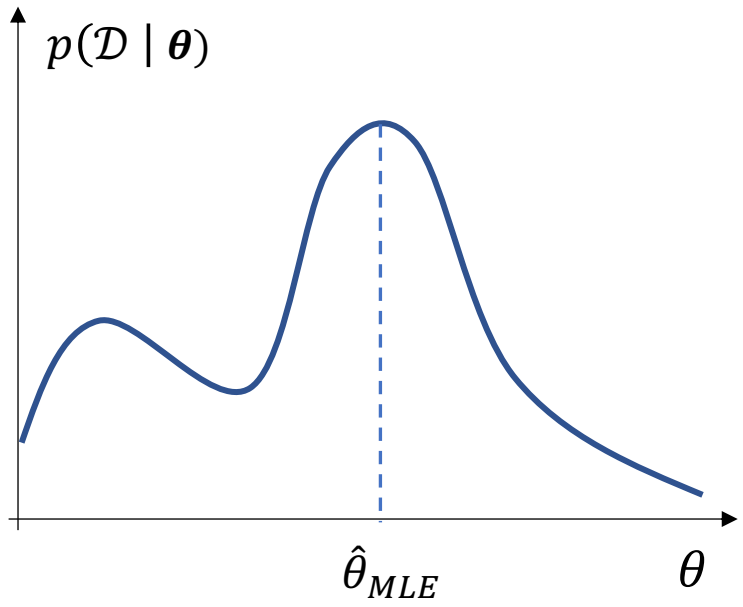
$$\hat{\boldsymbol{\theta}}_{MLE} = \arg\max_{\boldsymbol{\theta}} P(\mathcal{D} \mid \boldsymbol{\theta})$$



Maximum Likelihood Estimation (MLE)

❖ Maximize the *Likelihood* $P(\mathcal{D} \mid \boldsymbol{\theta})$ of $\mathcal{D}$ with respect to $\boldsymbol{\theta}$ → **Maximum Likelihood Estimate (MLE)**

$$\hat{\boldsymbol{\theta}}_{MLE} = \arg\max_{\boldsymbol{\theta}} P(\mathcal{D} \mid \boldsymbol{\theta})$$



$$L(\boldsymbol{\theta}; \mathcal{D}) = P(\mathcal{D} \mid \boldsymbol{\theta}) = \prod_{i=1}^{N} p(y_i \mid \boldsymbol{\theta})$$

$$\xrightarrow{Log} \quad \mathcal{L}(\boldsymbol{\theta}) = \log L(\boldsymbol{\theta}) = \sum_{i=1}^{N} \log\, p(y_i \mid \boldsymbol{\theta})$$

$\mathcal{L}(\boldsymbol{\theta})$: Log-Likelihood

$$\boxed{\hat{\boldsymbol{\theta}}_{MLE} = \arg\max_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) = \arg\max_{\boldsymbol{\theta}} \sum_{i=1}^{N} \log\, p(y_i \mid \boldsymbol{\theta})}$$

Note: hereafter
$\log == \ln$

Maximum Likelihood Estimation (MLE)

❖ **Optimization problem:** Difficult? It depends on the form of p (concave or not, highly multimodal)

$$\widehat{\boldsymbol{\theta}}_{MLE} = \arg\max_{\boldsymbol{\theta}} \mathcal{L}(\boldsymbol{\theta}) = \arg\min_{\boldsymbol{\theta}} [-\mathcal{L}(\boldsymbol{\theta})]$$

$$\boxed{\widehat{\boldsymbol{\theta}}_{MLE} = \arg\min_{\boldsymbol{\theta}} [-\mathcal{L}(\boldsymbol{\theta})]}$$

Negative Log-Likelihood ~ ***Loss function*** *(min)*

✓ Optimization of Negative Log-Likelihood is equivalent to perform **empirical risk minimization**

Optimization view of ML ↔ Probabilistic view of ML

Back to Logistic Regression and Loss function ….

- ❖ <u>Loss function</u>? → We want to learn a LR model (i.e., weight vector w and bias b of the linear function) such that the (conditional) likelihood of observing the given dataset $P(Y \mid \boldsymbol{X}, \boldsymbol{w}, b)$ is **maximized**
  - ❖ Note: our case, the *dataset* is the set of labels Y conditioned to the paired features $X, \{y^i, x^i\}_i^m$

- In other words: we want that, if the data labels $Y(X)$ were generated by our LR($\boldsymbol{w}, b$) model, it would be maximally *likely* to generate the same labels that we have in the dataset (compared to any other choice for the LR($\boldsymbol{w}, b$) model)

➢ In formal terms, the **likelihood function** is:

$$L(\boldsymbol{w}, b; \mathrm{Y}, \mathrm{X}) = \prod_{i=1}^{m} p\left(y^i \mid \boldsymbol{x}^i, \boldsymbol{w}, b\right)$$

○ Since there are only two discrete outcomes/labels, $\{0,1\}$, each $p\left(y^i \mid \boldsymbol{x}^i, \boldsymbol{w}, b\right)$ follows a **Bernoulli distribution:**

$$p(y \mid \boldsymbol{x}, \boldsymbol{w}, b) = \hat{y}^{y}(1-\hat{y})^{1-y}$$

where $\hat{y}$ is the prediction probability from the LR model: $\sigma(\boldsymbol{w}^T\boldsymbol{x} + b)$

Loss function = Negative Log-Likelihood function

- The **Negative Log-Likelihood function** → **Loss function**!:

$$\mathcal{L}(\boldsymbol{w}, b; \mathrm{Y}, \mathrm{X}) = -\log L(\boldsymbol{w}, b; \mathrm{Y}, \mathrm{X}) = -\sum_{i=1}^{N} \log p\left(y^i \mid \boldsymbol{x}^i, \boldsymbol{w}, b\right)$$

- Substituting $p(y \mid \boldsymbol{x}, \boldsymbol{w}, b) = \hat{y}^y (1-\hat{y})^{1-y}$

$$\mathcal{L}(\boldsymbol{w}, b; \mathrm{Y}, \mathrm{X}) = -\sum_{i=1}^{N} \log(\hat{y}^y (1-\hat{y})^{1-y})^i$$

$$= -\sum_{i=1}^{N} y^i \log \hat{y}^i + \left(1 - y^i\right) \log(1 - \hat{y}^i)$$

Cross-Entropy Loss

- Substituting $\hat{y} = \sigma(\boldsymbol{w}^T \boldsymbol{x} + b)$

$$\mathcal{L}_{CE}(\boldsymbol{w}, b; \mathrm{Y}, \mathrm{X}) = -\sum_{i=1}^{N} y^i \log \sigma(\boldsymbol{w}^T \boldsymbol{x}^i + b) + \left(1 - y^i\right) \log(1 - \sigma(\boldsymbol{w}^T \boldsymbol{x}^i + b))$$

Loss function = Negative Log-Likelihood function

$$\mathcal{L}_{CE}(\boldsymbol{w}, b; \mathrm{Y}, \mathrm{X}) = -\sum_{i=1}^{N} y^i \log \sigma(\boldsymbol{w}^T \boldsymbol{x}^i + b) + \left(1 - y^i\right) \log(1 - \sigma(\boldsymbol{w}^T \boldsymbol{x}^i + b))$$

- Substituting $\sigma\left(\boldsymbol{w}^T \boldsymbol{x}^i + b\right) = \dfrac{e^{\boldsymbol{w}^T \boldsymbol{x}^i + b}}{1 + e^{\boldsymbol{w}^T \boldsymbol{x}^i + b}}$ ....

Cross-Entropy Loss

$$\boxed{\mathcal{L}_{CE}(\boldsymbol{w}, b; \mathrm{Y}, \mathrm{X}) = \sum_{i=1}^{m} y^i (\boldsymbol{w}^T \boldsymbol{x}^i + b) - \ln\left[1 + e^{\boldsymbol{w}^T \boldsymbol{x}^i + b}\right]}$$

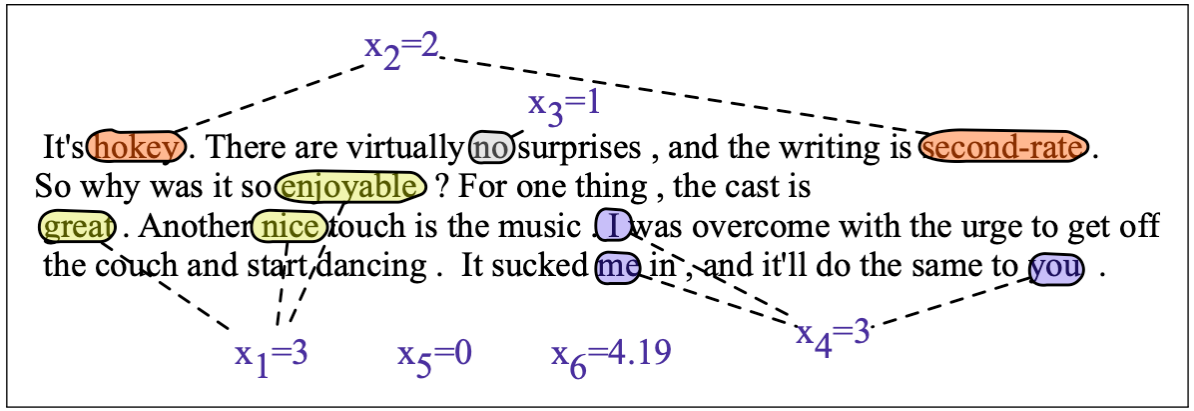
Using the cross-Entropy function as loss function, the following optimization problem is defined for LR training:

$$\boxed{\min_{\boldsymbol{w}, b} \mathcal{L}_{CE}(\boldsymbol{w}, b; \mathrm{Y}, \mathrm{X}) \equiv \min_{\boldsymbol{w}, b} \sum_{i=1}^{m} y^i (\boldsymbol{w}^T \boldsymbol{x}^i + b) - \ln\left[1 + e^{\boldsymbol{w}^T \boldsymbol{x}^i + b}\right]}$$

Loss function at work

Let's check that the loss function does the right job! →The sentiment analysis example considered before

Var	Definition	Value
x_1	count(positive lexicon words $\in$ doc)	3
x_2	count(negative lexicon words $\in$ doc)	2
x_3	$\begin{cases} 1 & \text{if "no"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	1
x_4	count(1st and 2nd pronouns $\in$ doc)	3
x_5	$\begin{cases} 1 & \text{if "!"} \in \text{doc} \\ 0 & \text{otherwise} \end{cases}$	0
x_6	ln(word count of doc)	$\ln(66) = 4.19$



➢ We want the loss to be smaller if the model's estimate is close to correct, and bigger if the model is confused.

- Let's suppose the true label y for the sentiment example is positive, i.e., y= 1
  - From before, for given $\boldsymbol{w}, b$: $p(y = 1 \mid \boldsymbol{x}, \boldsymbol{w}, b)$ = 0.7
    → Loss is small, 0.36

$$
\begin{aligned}
L_{\mathrm{CE}}(\hat{y}, y) &= -[y \log \sigma(\mathbf{w} \cdot \mathbf{x} + b) + (1-y) \log(1 - \sigma(\mathbf{w} \cdot \mathbf{x} + b))] \\
&= -[\log \sigma(\mathbf{w} \cdot \mathbf{x} + b)] \\
&= -\log(.70) \\
&= .36
\end{aligned}
$$

- Let's suppose the true label y for the sentiment example is negative, i.e., y= 0
  - From before, for given $\boldsymbol{w}, b$: $p(y = 0 \mid \boldsymbol{x}, \boldsymbol{w}, b)$ = 0.3
    → Loss is kinda large, 1.2

$$
\begin{aligned}
L_{\mathrm{CE}}(\hat{y}, y) &= -[y \log \sigma(\mathbf{w} \cdot \mathbf{x} + b) + (1-y) \log(1 - \sigma(\mathbf{w} \cdot \mathbf{x} + b))] \\
&= -[\log(1 - \sigma(\mathbf{w} \cdot \mathbf{x} + b))] \\
&= -\log(.30) \\
&= 1.2
\end{aligned}
$$

Logistic Regression: MCLE Optimization problem to solve

$$\widehat{\boldsymbol{w}}_{MCLE} = \arg\min_{\boldsymbol{w},b} \mathcal{L}_{CE}(\boldsymbol{w}, b; \mathrm{Y}, \mathrm{X}) = \arg\min_{\boldsymbol{w},b} \sum_{i=1}^{m} y^i(\boldsymbol{w}^T \boldsymbol{x}^i + b) - \ln\left[1 + e^{\boldsymbol{w}^T \boldsymbol{x}^i + b}\right]$$

Bad news: No closed-form solution to found the minimum of $\mathcal{L}_{CE}(\boldsymbol{w}, b; \mathrm{Y}, \mathrm{X})$

Good news: $\mathcal{L}_{CE}$ is a **convex** function of $\boldsymbol{w}$ (and $-\mathcal{L}_{CE}$ is a **concave** function)

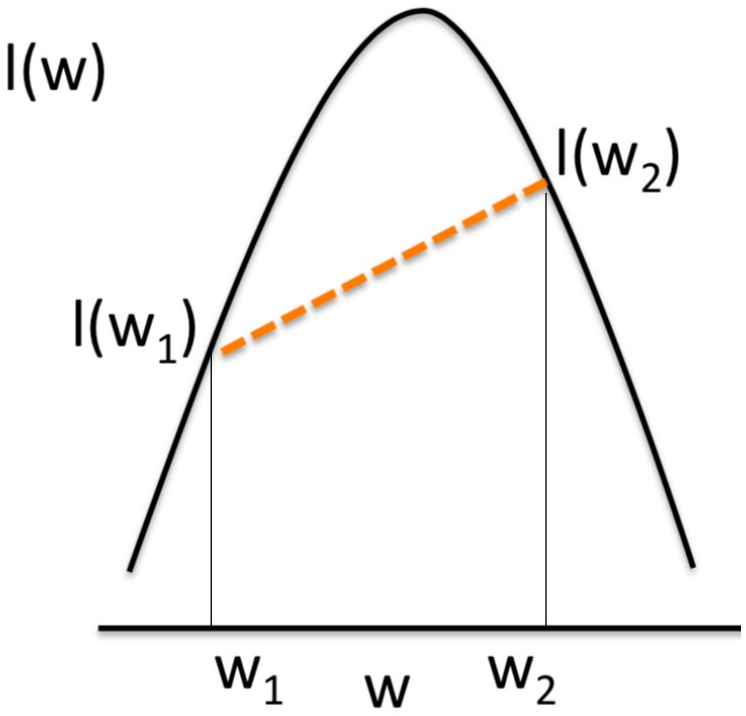
→ "Easy" to find min (max), polynomial time guarantees 👍

Why is it convex / concave?

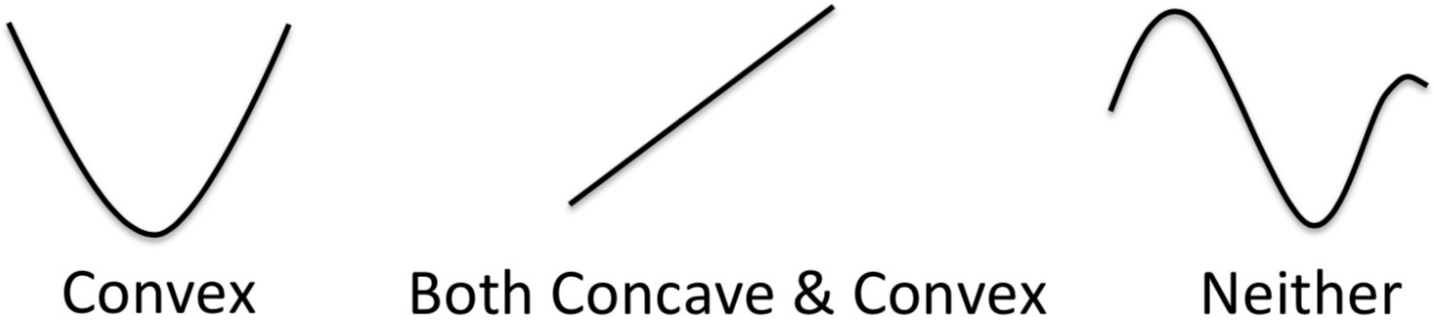
Linear combination of convex / concave functions

Let's solve it as a **Max** / **Concave** function !

Concave functions



A function $l(w)$ is called **concave** if the line joining any two points $l(w_1), l(w_2)$ on the function graph does not go *above* the function graph on the interval $[w_1, w_2]$



Theorem: For a convex (min) / concave (max) optimization problem, all *locally optimal* points are *globally optimal* (one, or infinite global optima)

Logistic Regression: MCLE Optimization problem (Max)

$$\hat{\mathbf{w}}_{MCLE} = \arg\max_{\mathbf{w}} \underbrace{\prod_{j=1}^{n} P(Y^{(j)} \mid X^{(j)}, \mathbf{w})}_{\text{(Conditional) Likelihood function, } \mathcal{L}(\boldsymbol{w};\boldsymbol{D})}$$

Log-Likelihood function

$$l(\mathbf{w}) \equiv \ln \prod_j P(y^j | \mathbf{x}^j, \mathbf{w})$$

$$= \sum_j \left[y^j (w_0 + \sum_i^d w_i x_i^j) - \ln(1 + exp(w_0 + \sum_i^d w_i x_i^j)) \right]$$

$$P(Y=0|\mathbf{X},\mathbf{w}) = \frac{1}{1 + exp(w_0 + \sum_i w_i X_i)}$$

$$P(Y=1|\mathbf{X},\mathbf{w}) = \frac{exp(w_0 + \sum_i w_i X_i)}{1 + exp(w_0 + \sum_i w_i X_i)}$$

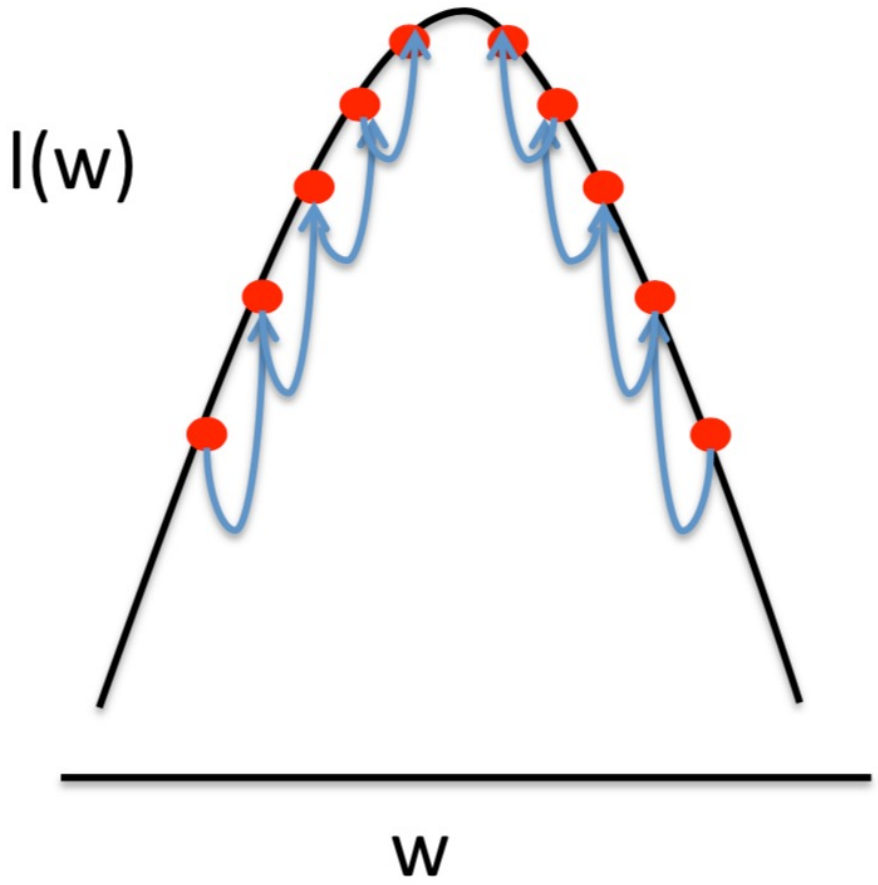
Modeling conditional probabilities using the **Logistic function**

$$\boxed{\boldsymbol{w}^* = \arg\max_{\boldsymbol{w}} l(\boldsymbol{w})}$$

Back to our optimization problem (concave)

Gradient Ascent Algorithm

Let's fix the step!



Initialize: Pick **w** at random

Gradient:

$$\nabla_{\mathbf{w}} l(\mathbf{w}) = [\frac{\partial l(\mathbf{w})}{\partial w_0}, \ldots, \frac{\partial l(\mathbf{w})}{\partial w_d}]'$$

Learning rate, η>0

Update rule:

$$\Delta \mathbf{w} = \eta \nabla_{\mathbf{w}} l(\mathbf{w})$$

$$w_i^{(t+1)} \leftarrow w_i^{(t)} + \eta \left. \frac{\partial l(\mathbf{w})}{\partial w_i} \right|_t$$

Gradient Ascent for logistic regression (MCLE)

Gradient ascent rule for w_0:

$$w_0^{(t+1)} \leftarrow w_0^{(t)} + \eta \left.\frac{\partial l(\mathbf{w})}{\partial w_0}\right|_t$$

$$l(\mathbf{w}) = \sum_j \left[y^j (w_0 + \sum_i^d w_i x_i^j) - \ln(1 + exp(w_0 + \sum_i^d w_i x_i^j)) \right]$$

$$\frac{\partial l(\mathbf{w})}{\partial w_0} = \sum_j \left[y^j - \underbrace{\frac{1}{1 + exp(w_0 + \sum_i^d w_i x_i^j)} \cdot exp(w_0 + \sum_i^d w_i x_i^j)} \right]$$

$$w_0^{(t+1)} \leftarrow w_0^{(t)} + \eta \sum_j [y^j - \hat{P}(Y^j = 1 \mid \mathbf{x}^j, \mathbf{w}^{(t)})]$$

Gradient Ascent for logistic regression (MCLE)

Gradient ascent algorithm: iterate until change < ε

$$w_0^{(t+1)} \leftarrow w_0^{(t)} + \eta \sum_j [y^j - \hat{P}(Y^j = 1 \mid \mathbf{x}^j, \mathbf{w}^{(t)})]$$

For i=1,…,d,

$$w_i^{(t+1)} \leftarrow w_i^{(t)} + \eta \sum_j x_i^j [y^j - \underbrace{\hat{P}(Y^j = 1 \mid \mathbf{x}^j, \mathbf{w}^{(t)})}_{\text{Predict what current weight thinks label Y should be}}]$$

repeat

Overfitting! → Regularization

- In principle, the loss can be brought to zero, given the convexity/concavity of the problem!
  - Overfitting the training data! (especially for small sized datasets)

→ Need to add **regularization** terms to avoid overfitting

$$\hat{\boldsymbol{w}}_{Lasso} = \arg\min_{\boldsymbol{w},b} \sum_{i=1}^{m} y^i\left(\boldsymbol{w}^T\boldsymbol{x}^i + b\right) - \ln\left[1 + e^{\boldsymbol{w}^T\boldsymbol{x}^i + b}\right] + \alpha \sum_{j=1}^{d} |w_i|$$

- **Lasso** (L1) prefers sparse solutions with some larger weights but many more weights set to zero (i.e., a few features_.

$$\hat{\boldsymbol{w}}_{Ridge} = \arg\min_{\boldsymbol{w},b} \sum_{i=1}^{m} y^i\left(\boldsymbol{w}^T\boldsymbol{x}^i + b\right) - \ln\left[1 + e^{\boldsymbol{w}^T\boldsymbol{x}^i + b}\right] + \beta \sum_{j=1}^{d} w_i^2$$

- **Ridge** (L2) regularization is easy to optimize because of its simple derivative, while L1 regularization is more complex. L2 prefers weight vectors with many small weights,

Logistic regression for more than two classes

Two classes:

$$p(y=1 \mid \boldsymbol{x}, \boldsymbol{w}) = \sigma(\boldsymbol{w}^T\boldsymbol{x}) = \frac{e^{\boldsymbol{w}^T\boldsymbol{x}}}{1+e^{\boldsymbol{w}^T\boldsymbol{x}}}$$

$$P(Y=1|\mathbf{X},\mathbf{w}) = \frac{exp(w_0+\sum_i w_i X_i)}{1+exp(w_0+\sum_i w_i X_i)}$$

$$p(y=0 \mid \boldsymbol{x}, \boldsymbol{w}) = 1-\sigma(\boldsymbol{w}^T\boldsymbol{x}) = \frac{1}{1+e^{\boldsymbol{w}^T\boldsymbol{x}}}$$

$$P(Y=0|\mathbf{X},\mathbf{w}) = \frac{1}{1+exp(w_0+\sum_i w_i X_i)}$$

***K* classes:**

$\boldsymbol{W} = [\boldsymbol{w}_1, \boldsymbol{w}_2, \cdots, \boldsymbol{w}_K]$ is a $(D+1)\times K$ **matrix**, one (column) D-dim vector of parameters per class, $w_K = \mathbf{0}$ (for identifiability)

$k < K$ $\quad p(y=k \mid \boldsymbol{x}, \boldsymbol{W}) = softmax(\boldsymbol{W}, \boldsymbol{x}, k) = \dfrac{e^{\boldsymbol{w}_k^T\boldsymbol{x}}}{1+\sum_{k\neq K} e^{\boldsymbol{w}_k^T\boldsymbol{x}}}$

$k = K$

$$p(y=K \mid \boldsymbol{x}, \boldsymbol{W}) = 1 - \sum_{k\neq K} \frac{e^{\boldsymbol{w}_k^T\boldsymbol{x}}}{1+\sum_{k\neq K} e^{\boldsymbol{w}_k^T\boldsymbol{x}}}$$

$$= \frac{1}{1+\sum_{k\neq K} e^{\boldsymbol{w}_k^T\boldsymbol{x}}}$$

$$P(Y=y_k|X) = \frac{\exp(w_{k0}+\sum_{i=1}^d w_{ki}X_i)}{1+\sum_{j=1}^{K-1}\exp(w_{j0}+\sum_{i=1}^d w_{ji}X_i)}$$

$$P(Y=y_K|X) = \frac{1}{1+\sum_{j=1}^{K-1}\exp(w_{j0}+\sum_{i=1}^d w_{ji}X_i)}$$